

CS/EE 147

GPU Computing & Programming

COMPREHENSIVE CUDA MIDTERM STUDY GUIDE

Prepared: April 21, 2026

Instructor: Nafis Mustakin
University of California, Riverside

This guide provides comprehensive coverage of all course topics with detailed explanations, code examples, and extensive practice problems.

TABLE OF CONTENTS

1. Course Overview & GPU Computing Fundamentals
2. Why GPUs? Architecture & Specialization
3. Computer Architecture Review (CPU Foundations)
4. CUDA C Programming Fundamentals
5. NVCC Compiler & Compilation Process
6. CUDA Parallelism Model (Grid, Block, Thread)
7. SIMT Execution Model & Warp Behavior
8. CUDA Memory Hierarchy (Detailed)
9. Memory Optimization Techniques
10. Parallel Computation Patterns
11. CUDA Streams & Asynchronous Programming
12. Performance Analysis & Profiling
13. Debugging & Common Pitfalls
14. Advanced Topics & Optimizations
15. Practice Problems & Solutions

1. COURSE OVERVIEW & GPU COMPUTING FUNDAMENTALS

1.1 Course Structure & Grading

Course Goal: Understand GPU architecture and learn to program GPUs for high performance. The course covers both the hardware (microarchitecture, SIMT execution) and software (CUDA programming, optimization).

Component	Weight	Details
Assignments	28%	4 assignments, 7% each
Final Project	25%	Major programming project
Exam 1 (Midterm)	25%	Focus: Architecture & programming basics
Exam 2 (Final)	20%	Focus: Advanced topics & optimization
Participation	2%	Class discussion & engagement

1.2 Assignment Policies

- **Slip Days:** 3 total per semester across all assignments
- **Late Penalty:** 15% per day late (cumulative)
- **Deadline:** Midnight Pacific Time on due date
- **Extensions:** None granted; use slip days wisely

1.3 Resources

- Course Website: nmustakin.github.io/teaching/CSEE147Sp26
- eLearn: Grades, announcements, Slack link
- Textbooks (optional): Kirk/Hwu, CUDA by Example, NVIDIA CUDA C Programming Guide
- Office Hours: Wednesday 1-2 PM @ WCH 110

2. WHY GPUS? ARCHITECTURE & SPECIALIZATION

2.1 Modern CPU Limitations

Modern CPU design is hitting fundamental walls:

Moore's Law Slowdown: Transistor density doubling has slowed significantly. We can't fit more transistors due to quantum effects and manufacturing limits.

Dennard Scaling End: Power per unit area is no longer constant. Voltage/frequency can't scale proportionally with transistor density, causing power consumption to explode.

Temperature Limits: Heat dissipation becomes the limiting factor, preventing higher clock speeds.

2.2 Specialization vs Generalization

When scaling hits limits, the solution is **specialization**—designing processors optimized for specific workloads. Examples include:

Device	Specialization	Use Case
GPU	Parallel throughput	Graphics, AI/ML, scientific computing
TPU (Tensor)	Matrix operations	Neural network inference/training
ASIC (Bitcoin)	Hash computation	Cryptocurrency mining
FPGA	Custom logic	Flexible, reconfigurable

2.3 CPU vs GPU Design Philosophy

Aspect	CPU (Latency-Optimized)	GPU (Throughput-Optimized)
Primary Goal	Minimize latency of individual operations	Maximize total throughput
Core Count	2-64 powerful cores	Thousands of simple cores
ALU Design	Complex, sophisticated	Simple, energy-efficient
Cache Size	Large L1/L2/L3 caches	Small L1, shared memory
Branch Prediction	Advanced predictors	None or very simple
Data Forwarding	Yes, sophisticated	No or minimal
Instruction Level	Superscalar, out-of-order	SIMD, in-order (mostly)
Best For	Sequential code, low latency	Parallel code, high throughput
Typical Speed	10x+ faster on serial	10x+ faster on parallel

2.4 CPU Architecture Details

Latency-Oriented Design: CPUs minimize latency (time for one operation to complete) through:

- Powerful ALUs: Complex hardware for sophisticated operations
- Large Caches: L1/L2/L3 convert long-latency memory accesses to short-latency cache hits
- Branch Prediction: Speculative execution avoids pipeline stalls on branches
- Data Forwarding: Results available immediately to dependent instructions

This costs chip area and power but pays off for sequential, latency-sensitive code.

2.5 GPU Architecture Details

Throughput-Oriented Design: GPUs maximize throughput (operations per unit time) through:

- Small Caches: More chip area for compute units instead of cache
- Simple Control: No branch prediction or data forwarding (avoids stalls by having many threads)
- Energy-Efficient ALUs: Simple hardware uses less power
- Massive Parallelism: Thousands of threads tolerate latency through context switching

Trade-off: Single-thread performance is poor, but aggregate throughput is excellent.

3. COMPUTER ARCHITECTURE REVIEW

3.1 Virtual Memory

Virtual memory allows each process to have its own isolated address space, much larger than physical memory. The OS and MMU (Memory Management Unit) translate virtual addresses to physical addresses.

Key Concepts:

- **Virtual Address Space:** Each process sees 256 TB of address space (for 64-bit systems)
- **Pages:** Virtual memory divided into fixed-size chunks (typically 4 KB)
- **Page Table:** Maps virtual pages to physical frames in DRAM
- **Page Miss/Swap:** If page not in physical memory, OS fetches from disk (very slow)
- **Benefit:** Processes isolated; physical memory shared; can run processes larger than RAM

3.2 Process vs Thread

Process: Heavy-weight execution context. Each process has its own:

- Virtual address space
- Open file descriptors
- Process-level resources

Context switching between processes is expensive (TLB flush, cache invalidation).

Thread: Light-weight execution context within a process. Threads share:

- Same virtual address space
- Global variables, heap

Each thread has private:

- Stack
- Register state
- Program counter

Context switching between threads is faster (no virtual memory flush needed).

3.3 CPU Memory Hierarchy

Modern CPUs have multiple cache levels to hide main memory latency:

Level	Size	Latency	Purpose
Registers	~1 KB	~0.5 ns	CPU operands
L1 Cache	32 KB	~4 ns	Fastest caching
L2 Cache	256 KB	~11 ns	Per-core cache
L3 Cache	8-20 MB	~40 ns	Shared across cores
Main Memory	GB	~100-200 ns	DRAM
Disk	TB	~10 ms	Slow storage

Design Principle: Each level is a cache for the level below. L1 caches data from L2, which caches from L3, etc. The goal is to keep frequently-accessed data in fast, small caches.

3.4 Instruction Set Architecture (ISA)

ISA is the contract between hardware and software. It defines:

- **Instruction Formats:** How bits are organized (opcode, registers, immediates)
- **Instruction Semantics:** What each instruction does
- **Register Set:** Available registers and their purposes
- **Memory Model:** How memory is addressed and accessed

Example (MIPS R-format): add \$r8, \$r17, \$r18

Encoding: [opcode:6][rs:5][rt:5][rd:5][shamt:5][funct:6]
= [000000][10001][10010][01000][00000][100000] = 0x02324020

4. CUDA C PROGRAMMING FUNDAMENTALS

4.1 CUDA Programming Model Overview

CUDA is a heterogeneous computing platform where CPUs and GPUs work together:

Host Code (CPU): Manages memory, launches kernels, handles I/O

Device Code (GPU): Massively parallel computations

Programs are written in C with CUDA extensions. NVCC (NVIDIA CUDA Compiler) separates host and device code, compiling them with different compilers.

4.2 CUDA Function Qualifiers

Qualifier	Executes On	Callable From	Notes
__global__	Device (GPU)	Host (CPU)	Kernel function; must return void
__device__	Device	Device only	Helper/utility functions
__host__	Host (CPU)	Host only	Normal C function; optional
__host__ __device__	Both	Both	Compiled for both (templates)

4.3 Kernel Execution Model

A kernel is launched from host code and executed by many threads on the GPU. Syntax: `kernel_name<<<gridDim, blockDim>>>(args)`

Example:

```
__global__ void vecAddKernel(float* A, float* B, float* C, int n) {
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if (i < n) C[i] = A[i] + B[i];
}
```

```
// Host code
vecAddKernel<<<ceil(n/256.0), 256>>>(d_A, d_B, d_C, n);
```

Each thread computes one element. `blockIdx.x` identifies block, `threadIdx.x` identifies thread within block.

4.4 Important CUDA APIs

Memory Allocation:

- `cudaMalloc(void** devPtr, size_t size)` - Allocate device memory
- `cudaFree(void* devPtr)` - Free device memory
- `cudaHostAlloc(..., cudaHostAllocDefault)` - Pinned host memory
- `cudaFreeHost(void* ptr)` - Free pinned memory

Data Transfer:

- `cudaMemcpy(dst, src, size, cudaMemcpyHostToDevice)` - Copy H to D
- `cudaMemcpy(dst, src, size, cudaMemcpyDeviceToHost)` - Copy D to H
- `cudaMemcpy(dst, src, size, cudaMemcpyDeviceToDevice)` - Copy D to D

Synchronization:

- `cudaDeviceSynchronize()` - Block until GPU finishes all work
- `__syncthreads()` - Synchronize threads within a block (device code)

5. NVCC COMPILER & COMPILATION PROCESS

5.1 NVCC Overview

NVCC is NVIDIA's compiler for CUDA C. It:

1. Parses .cu files for CUDA constructs
2. Compiles device code to PTX (Parallel Thread eXecution, virtual ISA)
3. Forwards host code to system C compiler (g++, clang, msvc)
4. Links device and host code into final executable

Key Point: Only .cu files are parsed for CUDA directives. To use CUDA in .cc files, rename to .cu or use -x cu flag.

5.2 Compilation Flags

Device Code Flags:

- -G - Include device debugging symbols (disables optimizations)
- -O3 - Enable device optimizations (default for release builds)
- -lineinfo - Include line information for profilers
- -arch=compute_XY - Compile for compute capability XY
- -code=sm_ZW - Generate code for specific GPU

Host Code Flags:

- -x cu - Treat all input files as .cu
- -Xcompiler -flag - Forward flag to host compiler
- Example: -Xcompiler -fopenmp (forward OpenMP flag)

Common Combinations:

- Debug: nvcc -G -Xcompiler -g program.cu
- Profile: nvcc -O3 -lineinfo program.cu

5.3 PTX (Parallel Thread eXecution)

PTX is a virtual intermediate representation used by NVIDIA. Benefits:

- **Binary Compatibility:** Code compiled to PTX works on multiple GPU architectures
- **JIT Compilation:** PTX compiled to machine code by GPU driver at runtime
- **Infinite Registers:** PTX supports unlimited virtual registers (hardware maps to physical)
- **Architecture Independence:** New GPU architectures get new PTX backends without recompiling user code

Example PTX syntax:

```
.reg .f32 f1, f2;
ld.global.f32 f1, [ptr];
add.f32 f2, f1, 1.0;
st.global.f32 [ptr], f2;
```

6. CUDA PARALLELISM MODEL (GRID, BLOCK, THREAD)

6.1 Grid-Block-Thread Hierarchy

CUDA organizes threads in a three-level hierarchy:

Grid: Collection of blocks executing the same kernel. Blocks are independent and can run in any order.

- Can be 1D, 2D, or 3D
- Specified at kernel launch as `gridDim` (e.g., `dim3 gridDim(64, 32, 1)`)

Block: Group of threads that share shared memory and can synchronize with `__syncthreads()`.

- Can be 1D, 2D, or 3D
- Specified at kernel launch as `blockDim`
- Maximum 1024 threads per block (hardware limit)
- All threads in block must fit on one SM (streaming multiprocessor)

Thread: Individual execution unit. Each thread has:

- Private registers and local memory
- Unique `threadIdx` within block
- Access to block's shared memory

6.2 Thread Indexing

Each thread knows its position via built-in variables:

threadIdx: (x, y, z) position within block (0-indexed)

blockIdx: (x, y, z) position of block within grid

blockDim: (x, y, z) dimensions of block

gridDim: (x, y, z) dimensions of grid

Computing Global Index (1D):

```
int globalIdx = threadIdx.x + blockDim.x * blockIdx.x;
```

Computing Global Index (2D):

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
int globalIdx = row * width + col; // Row-major layout
```

6.3 Configuring Grid and Block Dimensions

Strategy 1: Fixed Block Size, Computed Grid Size

```
dim3 blockDim(256, 1, 1);
dim3 gridDim((n + blockDim.x - 1) / blockDim.x, 1, 1);
kernel<<<gridDim, blockDim>>>(...);
```

Typical block sizes: 128, 256, or 512 threads (power of 2, good for coalescing)

Strategy 2: 2D Grid for Image Processing

```
dim3 blockDim(16, 16, 1); // 256 threads per block
dim3 gridDim((width + 15) / 16, (height + 15) / 16, 1);
imageKernel<<<gridDim, blockDim>>>(...);
```

6.4 Example: 2D Image Processing Kernel

```
__global__ void blurKernel(uchar* in, uchar* out, int w, int h, int f) {
    // f = filter radius
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < h && col < w) {
        float sum = 0.0f;
        int count = 0;
```

```
for (int dy = -f; dy <= f; dy++) {
    for (int dx = -f; dx <= f; dx++) {
        int y = row + dy, x = col + dx;
        if (y >= 0 && y < h && x >= 0 && x < w) {
            sum += in[y * w + x];
            count++;
        }
    }
}
out[row * w + col] = (uchar)(sum / count);
}
```

7. SIMT EXECUTION MODEL & WARP BEHAVIOR

7.1 What is a Warp?

A **warp** is the fundamental execution unit on NVIDIA GPUs. NVIDIA groups 32 consecutive threads into a warp that executes in lockstep on SIMD (Single Instruction Multiple Data) hardware.

Key Points:

- Threads 0-31 form warp 0, threads 32-63 form warp 1, etc.
- All threads in a warp execute the same instruction
- All threads progress synchronously (mostly)
- If a thread stalls, the entire warp stalls
- Terminology: AMD calls them "wavefronts" (64 threads)

Design Rationale: Grouping threads into warps allows GPU hardware to be simple. Instead of 1024 independent instruction decoders, GPU needs only 32 (one per warp).

7.2 Warp Divergence

When threads in the same warp take different code paths, the hardware **serializes** them. This is called **branch divergence** or **warp divergence**.

Example:

```
__global__ void divergeKernel() {
    if (threadIdx.x < 16) {
        // Path A: threads 0-15 execute this
        doWorkA();
    } else {
        // Path B: threads 16-31 execute this
        doWorkB();
    }
}
```

What Happens:

1. Hardware executes Path A with all threads enabled (0-31)
2. Threads 0-15 do useful work; threads 16-31 are masked off (predicated) and idle
3. Hardware then executes Path B with predicate inverted
4. Threads 16-31 do work; threads 0-15 are idle

Result: Warp execution time = time(Path A) + time(Path B), worst case 2x slowdown!

Minimizing Divergence:

- Use blockIdx/threadIdx in conditions (consecutive threads follow same path)
- Avoid data-dependent branches
- Reorganize data so threads with similar values are consecutive

7.3 Programmer's View vs Hardware Reality

What Programmer Sees (MIMD):

Thousands of independent scalar threads, each with its own:

- Program counter
- Register file
- Local memory

Each thread can take independent control flow paths.

What Hardware Does (SIMD):

32 scalar threads grouped into one warp. All execute:

- Same instruction from same program counter
- On different data from different register files

Different control flow is achieved through predication, not actual divergence.

Takeaway: CUDA allows flexible programming (MIMD), but understanding SIMD execution (warps) is essential for performance.

8. CUDA MEMORY HIERARCHY (DETAILED)

8.1 Memory Types and Their Properties

Memory Type	Scope	Lifetime	Declaration	Speed	Size
Registers	Thread	Thread	auto/int x	Very Fast	Limited
Local Memory	Thread	Thread	int arr[4]	Slow	Limited
Shared	Block	Block	__shared__	Very Fast	96 KB/block
Global	Grid/App	App	__device__	Slow	GB
Constant	Grid/App	App	__constant__	Medium	64 KB
Texture	Grid/App	App	texture	Medium	Cache

8.2 Registers

Speed: Fastest memory on GPU (no latency)

Size: ~256 KB per SM (shared among all threads on SM)

Scope: Private to each thread

Lifetime: Dies when thread ends

How it Works: Compiler allocates registers automatically for local variables. If too many registers needed, spills to local memory (slow!).

Optimization: Minimize register usage (fewer registers per thread = more threads per SM)

8.3 Shared Memory

Speed: ~100x faster than global memory

Size: 48-96 KB per block (fixed at compile time)

Scope: All threads in block can read/write

Lifetime: Dies when block finishes

Synchronization: Use __syncthreads() to ensure all threads see writes

Declaration:

```
__global__ void kernel() {  
    __shared__ float s_data[256];  
    s_data[threadIdx.x] = global_data[threadIdx.x];  
    __syncthreads(); // Wait for all threads to load  
    // Now all threads can safely read s_data from any index  
}
```

Bank Conflicts: Shared memory is divided into banks. If multiple threads access same bank, access is serialized. Design access patterns to avoid bank conflicts.

8.4 Global Memory

Speed: 100-200 cycles latency

Size: GB (typical: 6-80 GB on modern GPUs)

Scope: All threads can read/write

Lifetime: Persists until kernel ends (or cudaFree)

Cache: Cached in L1/L2 caches

Bandwidth: High (1-2 TB/s on modern GPUs) if accessed efficiently

Coalescing Requirements: Consecutive threads should access consecutive addresses for good performance. GPU hardware combines individual 4-byte accesses into 128-byte cache line loads.

8.5 Constant Memory

Speed: Fast when all threads read same value (cached)

Size: 64 KB

Scope: Read-only from device code

Declaration: `__device__ __constant__ float c_value;`

Initialization: `cudaMemcpyToSymbol(c_value, host_ptr, sizeof(float));`

Use Case: Broadcast same value to all threads (e.g., filter coefficients)

9. MEMORY OPTIMIZATION TECHNIQUES

9.1 Memory Coalescing

Core Idea: GPU memory hardware is optimized for accessing consecutive addresses. If threads 0-31 (one warp) access addresses [0,4,8,12,...,124], that's 32 separate transactions. But if they access [0,1,2,3,...,31], that's one 128-byte cache line.

Coalesced Access Pattern (GOOD):

```
int idx = threadIdx.x; // Each thread has unique index
float value = global_array[idx]; // Threads 0-31 read 0-31
// Result: One memory transaction for entire warp
```

Non-Coalesced Access Pattern (BAD):

```
int idx = threadIdx.x * 1024; // Threads spread out
float value = global_array[idx]; // Threads 0-31 read 0, 1024, 2048, ...
// Result: 32 separate memory transactions
```

For 2D Arrays: Access elements in row-major order:

```
// GOOD
int idx = row * width + col;
float value = data[idx];
// AVOID
int idx = row + col * height; // Column-major access
```

9.2 Tiling/Blocking Pattern

Problem: Global memory has high latency and limited bandwidth. If computation requires many global memory accesses, GPU stalls waiting for data.

Solution: Load data into shared memory (on-chip cache). Process tiles while data is in fast memory. Reduces global memory bandwidth by 10-100x.

Example: Matrix Multiplication

Without tiling: Each of $N \times N$ output elements requires $2N$ global memory loads. Total: $2N^3$ loads.

With tiling (tile size T): Each output tile loads $T \times N$ elements from memory for T^2 output elements. Total: $(2N^3) / T$ loads, T times reduction!

Implementation Pattern:

```
__global__ void tiledKernel(float* A, float* B, float* C, int N) {
    __shared__ float s_A[TILE_SIZE][TILE_SIZE];
    __shared__ float s_B[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < N; t += TILE_SIZE) {
        // Load tile from global to shared
        s_A[threadIdx.y][threadIdx.x] = A[row * N + (t + threadIdx.x)];
        s_B[threadIdx.y][threadIdx.x] = B[(t + threadIdx.y) * N + col];
        __syncthreads();

        // Compute with tile
        for (int i = 0; i < TILE_SIZE; i++)
            sum += s_A[threadIdx.y][i] * s_B[i][threadIdx.x];
        __syncthreads();
    }
    C[row * N + col] = sum;
}
```


9.3 Bank Conflicts in Shared Memory

Shared memory is divided into 32 banks (one per thread in ideal case). If two threads in same warp access different addresses in same bank, access is serialized.

Bank Conflict Example:

Bank width = 4 bytes. Addresses 0-3 in bank 0, 4-7 in bank 1, ..., 128-131 in bank 0 (wraps).

If threadIdx.x accesses s_data[2 * threadIdx.x], threads access addresses [0, 2, 4, 6, ...]. With 4-byte words: banks [0, 0, 1, 1, ...].

Threads 0&1 conflict on bank 0!

Avoid by Padding:

```
__shared__ float s_data[TILE_SIZE + 1]; // +1 to avoid conflicts
```

10. PARALLEL COMPUTATION PATTERNS

10.1 Reduction Pattern

Problem: Combine N values into 1 (e.g., sum all elements).

Naïve Approach: One thread sums all N values serially. Very slow.

Parallel Reduction: Tree of additions. Use $\log(N)$ stages.

Stage 1: Threads pair up; each adds two values (N/2 adds)

Stage 2: Remaining threads pair up (N/4 adds)

...

Stage $\log(N)$: One thread has final result

Implementation (Shared Memory Reduction):

```
__global__ void reduceKernel(int* input, int* output, int n) {
    __shared__ int s_data[BLOCK_SIZE];
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + threadIdx.x;

    // Load into shared memory
    s_data[tid] = (gid < n) ? input[gid] : 0;
    __syncthreads();

    // Reduction in shared memory
    for (int stride = blockDim.x / 2; stride > 0; stride /= 2) {
        if (tid < stride)
            s_data[tid] += s_data[tid + stride];
        __syncthreads();
    }

    // Write result
    if (tid == 0)
        output[blockIdx.x] = s_data[0];
}
```

Complexity: $O(\log n)$ parallel time, $O(n)$ work. Efficient!

10.2 Scan/Prefix Sum

Problem: Compute prefix sums: $\text{output}[i] = \text{sum of input}[0..i]$

Work Efficient Parallel Algorithm: Blelloch scan ($O(n)$ work, $O(\log n)$ time)

Up-sweep phase: Reduce to sum

Down-sweep phase: Distribute partial sums

Result: Logarithmic parallel time with linear work.

10.3 Stencil Pattern

Problem: Each output depends on a neighborhood of inputs (e.g., blur, convolution)

Challenge: Ghost cells. Boundary threads need values from neighboring blocks.

Solution: Load halo (ghost) cells into shared memory:

Interior threads: Load 1 element each

Boundary threads: Load 1 normal + 1 halo element

Synchronize, then compute.

11. CUDA STREAMS & ASYNCHRONOUS PROGRAMMING

11.1 What are Streams?

Definition: A stream is a sequence of commands executed in order on the GPU. Commands in different streams can execute concurrently.

Default Stream (Stream 0): If no stream specified, all operations use default stream. All operations serialized.

Multiple Streams: Create streams to overlap computation and data transfer:

Stream A: Kernel 1

Stream B: cudaMemcpy (D to H) overlaps with Kernel 1

Stream C: Kernel 2 overlaps with both

API:

```
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

kernel1<<<grid, block, 0, stream1>>>(...);
kernel2<<<grid, block, 0, stream2>>>(...);
cudaMemcpyAsync(..., cudaMemcpyDeviceToHost, stream1);

cudaStreamSynchronize(stream1); // Wait for stream1
cudaStreamDestroy(stream1);
```

11.2 Pinned Memory

Problem: cudaMemcpy requires pinned (page-locked) memory on host side. If given regular memory, driver must first copy to pinned buffer (extra overhead).

Solution: Allocate host memory as pinned:

```
float* h_data;
cudaHostAlloc(&h_data, size, cudaHostAllocDefault);
cudaMemcpyAsync(d_data, h_data, size, cudaMemcpyHostToDevice, stream);
cudaFreeHost(h_data);
```

Trade-off: Pinned memory faster for transfers, but reduces system memory available. Use sparingly.

12. PERFORMANCE ANALYSIS & PROFILING

12.1 Profiling Tools

Nsight Compute (nv-nsight-compute-cli):

Kernel-level profiling. Measures:

- SM utilization (% SMs with active warps)
- Warp efficiency (useful vs idle threads)
- Memory bandwidth utilization
- Cache hit rates
- Instruction throughput

Nsight Systems (nsys):

System-level profiling. Measures:

- Data transfer rates (PCIe bandwidth)
- Kernel launch overhead
- GPU utilization timeline
- CPU-GPU synchronization points

Usage:

```
nsys profile ./my_program
nv-nsight-compute-cli --set full ./my_program
```

12.2 Common Performance Issues

- 1. Low Warp Efficiency:** Many threads idle due to divergence or unequal work. Reorganize data/control flow to reduce divergence.
- 2. Low SM Utilization:** Not enough active warps on SMs. Launch larger grid, use fewer registers per thread.
- 3. Memory Bandwidth Bottleneck:** Many uncoalesced accesses or strided patterns. Use shared memory, tile data, improve access patterns.
- 4. Global Memory Stalls:** Kernels waiting on memory. Profile with Nsight Compute to check. Consider cache optimization, coalescing, data reuse.
- 5. PCIe Bottleneck:** Data transfers too slow. Use pinned memory, overlap with kernels, minimize transfers.

13. DEBUGGING & COMMON PITFALLS

13.1 Debugging Tools

cuda-memcheck: Detects memory errors without recompilation.

```
cuda-memcheck ./my_program
```

Finds:

- Out-of-bounds accesses
- Memory leaks
- Race conditions
- Uninitialized memory reads

cuda-gdb: GDB with device code support.

```
nvcc -G program.cu # Compile with debug info
```

```
cuda-gdb ./a.out
```

printf in Kernels:

```
__global__ void debugKernel() {  
    printf("Thread %d, value = %d\n", threadIdx.x, value);  
}
```

13.2 Common Pitfalls

Pitfall 1: Forgetting __syncthreads()

Threads might read stale shared memory before all threads write. Always sync after writes.

Pitfall 2: Race Conditions

Multiple threads writing same global memory location without atomics. Use atomicAdd(), etc.

Pitfall 3: Divergence

Data-dependent branches cause warp divergence. Use control flow that depends on blockIdx/threadIdx.

Pitfall 4: Not Checking Return Values

cuda* functions return errors. Check with cudaGetLastError().

Pitfall 5: Shared Memory Bank Conflicts

Multiple threads accessing same bank serialize. Pad arrays or use different access patterns.

Pitfall 6: Uninitialized Device Memory

cudaMalloc doesn't zero memory. Use cudaMemset or initialize explicitly.

14. ADVANCED TOPICS & OPTIMIZATIONS

14.1 Atomic Operations

Atomics allow safe concurrent writes to same memory location. Expensive but correct.

Common Atomics:

```
atomicAdd(&var, val); // var += val, atomic
atomicSub(&var, val); // var -= val
atomicExch(&var, val); // var = val, return old
atomicCAS(&var, compare, val); // compare-and-swap
```

Cost: Can be 10-100x slower than regular loads/stores. Use only when necessary.

14.2 Occupancy

Definition: Occupancy = (active warps per SM) / (max warps per SM)
Modern GPUs: max 32-64 warps per SM (depends on architecture)

Factors Limiting Occupancy:

1. Registers per thread: More registers = fewer threads per SM
2. Shared memory: More allocated = fewer blocks per SM
3. Threads per block: Must divide max threads per SM evenly

Goal: 50-100% occupancy for good latency hiding. Measure with nvidia-smi or profilers.

14.3 Intrinsic Functions

GPU has specialized fast functions for common operations:

```
__sinf(x) - Fast sine
__expf(x) - Fast exponential
__fdividef(x, y) - Fast division (less accurate)
__popc11(x) - Count bits
```

Lower precision but much faster. Use when accuracy allows.

15. PRACTICE PROBLEMS & SOLUTIONS

15.1 Conceptual Questions

Q1: What is a warp and why do GPUs use warps instead of executing threads independently?

A: A warp is 32 threads executed in lockstep on SIMD hardware. This simplifies hardware—instead of 32 independent decoders, GPU needs 1 decoder per warp. Reduces chip complexity.

Q2: Explain warp divergence with an example. How does it impact performance?

A: When threads in a warp follow different code paths, hardware serializes them using predicates. Thread 0-15 execute if-branch while 16-31 are masked off, then vice versa for else-branch. Result: serial execution, 2x slowdown worst case.

Q3: Why are GPUs bad at sequential code but good at parallel code?

A: GPUs optimize for throughput (many operations) not latency (single operation speed). Low clock, simple ALU, small caches. But with 1000s of threads, overall throughput is high. CPUs are opposite.

Q4: What is memory coalescing? Why does it matter?

A: When consecutive threads access consecutive addresses, GPU memory hardware combines them into one large transaction. Uncoalesced accesses require many separate transactions. Coalescing improves bandwidth 10-100x.

Q5: Explain shared memory. When should you use it?

A: Shared memory is on-chip cache, 100x faster than global. Each block has its own (48-96 KB). Use for reducing global memory traffic, tiling, communication between threads.

Q6: What are the constraints on CUDA blocks and grids?

A: Max 1024 threads per block. Block must fit on one SM. Grid dimensions limited by compute capability. Typical: 256-512 threads/block for throughput.

Q7: How do you calculate thread indices for 2D image processing?

A: $\text{row} = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$; $\text{col} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$; $\text{idx} = \text{row} * \text{width} + \text{col}$;

Q8: What is register pressure and how does it affect occupancy?

A: High register usage per thread limits threads per SM (occupancy). If kernel uses 64 registers and SM has 256K registers, only 4K registers per thread = 4 threads max. Reduce registers to improve occupancy.

15.2 More Practice Questions

1. Describe the phases of CUDA code compilation (nvcc, PTX, JIT).
2. What is the difference between `__syncthreads()` and `cudaDeviceSynchronize()`?
3. How would you optimize a kernel that has poor global memory utilization?
4. Explain the reduction pattern for summing N elements. What's the time complexity?
5. When would you use pinned memory? What are the tradeoffs?
6. What causes bank conflicts in shared memory? How do you avoid them?
7. Write a kernel that squares each element of an array.
8. Write a reduction kernel that sums all elements in a block.
9. How do you measure GPU kernel performance with profilers?
10. What is the typical PCIe bandwidth? How does it compare to GPU memory bandwidth?
11. Explain the grid-block-thread hierarchy. Why does CUDA use this 3-level structure?
12. What are the differences between local memory, shared memory, and global memory?
13. Why is loop unrolling beneficial on GPUs?
14. How do CUDA streams enable asynchronous execution?
15. What determines GPU occupancy and why does it matter?

STUDY TIPS FOR THE MIDTERM

- **Understand Warp Execution:** This is the #1 concept. Know that 32 threads execute in lockstep, what happens on divergence, and why it matters.
- **Master Thread Indexing:** Practice calculating indices for 1D, 2D, and 3D grids. This is fundamental to writing kernels.
- **Memory Matters:** Understand the memory hierarchy deeply. Coalescing, shared memory, and global memory access patterns dominate performance.
- **Divergence is Bad:** Minimize branches that depend on thread IDs if thread IDs differ within warp. Use blockIdx/threadIdx intelligently.
- **Synchronization:** Know when to use `__syncthreads()` and why. Common source of bugs.
- **Review Examples:** Understand vector addition, matrix multiply, reduction, scan. These are canonical patterns.
- **Do Calculations:** Practice hand-calculating grid/block configs, thread indices, coalescing patterns. Don't just memorize.
- **Profiling Mindset:** Think about SM utilization, memory bandwidth, warp efficiency. Performance isn't just about correctness.

GOOD LUCK ON YOUR MIDTERM!